FUNDAMENTOS DE C++

Variables, tipos de datos, asignación, constantes y operadores

Tipo de recurso	Documentación técnica para repositorio
Lenguaje	C++ (ejemplos introductorios)
Objetivo	Comprender los conceptos y practicar con ejercicios progresivos

Guía de estudio • explicación conceptual • ejemplos comentados • ejercicios sin resolver

Los ejemplos están pensados para un nivel inicial. Puedes compilarlos con un compilador compatible con C++ moderno.

Contenido

Sección	Tema	Página aprox.
1.	Variables	3
1.1	¿Qué es una variable?	3
1.2	¿Para qué sirven las variables?	3
1.3	Características de una variable	4
2.	Tipos de datos	5
3.	Asignación de variables	6
4.	Constantes	7
5.	Operadores	8
5.1	Operadores matemáticos	8
5.2	Operadores de asignación	9
5.3	Operadores relacionales	10
5.4	Operadores lógicos	11
6.	Ejercicios integradores	12
7.	Buenas prácticas y errores comunes	13

Nota: la numeración de páginas del índice es orientativa; puede variar ligeramente según el visor o la impresión.

1. Variables

1.1. ¿Qué es una variable?

Una variable es un espacio identificado por un nombre que permite almacenar un dato durante la ejecución de un programa. En C++, cada variable tiene un tipo de dato que determina qué clase de valor puede guardar y qué operaciones se pueden realizar con él.

```
int edad = 18;
double promedio = 87.5;
char inicial = 'A';
```

En este ejemplo, edad guarda un entero, promedio guarda un número con parte decimal y inicial guarda un carácter. El valor de una variable puede cambiar si el programa le asigna otro valor.

1.2. ¿Para qué sirven las variables?

Permiten conservar datos para utilizarlos después, realizar cálculos, recibir información del usuario y tomar decisiones. Sin variables, sería difícil crear programas que respondan a datos diferentes.

```
int precio = 1200;
int cantidad = 3;
int total = precio * cantidad;
// total contiene 3600
```

El programa calcula el costo total a partir del precio y la cantidad. Si cambia cualquiera de esos datos, el cálculo puede producir otro resultado.

1.3. Características de una variable

• Nombre: identificador que permite referirse al dato, por ejemplo, `cantidad`.

• Tipo: define la clase de dato, por ejemplo, `int` o `double`.

• Valor: contenido almacenado en un momento determinado.

• Declaración: instrucción con la que se indica el tipo y el nombre.

• Inicialización: asignación de un valor inicial al declarar la variable.

Reglas básicas para nombrar variables en C++:

• El nombre puede contener letras, números y guion bajo (_), pero no espacios.

• No debe comenzar con un número.

• No puede ser una palabra reservada del lenguaje, como `int` o `while`.

• C++ distingue entre mayúsculas y minúsculas: `nota` y `Nota` son identificadores distintos.

• Es recomendable usar nombres descriptivos, como `cantidadLibros`.

Ejemplo comentado

```
int nota = 80;       // Declaración e inicialización
nota = 95;           // La variable cambia de valor
cout << nota;        // Muestra 95
```

Observa que el tipo se escribe al declarar la variable. Para cambiar su valor después, basta con escribir el nombre, el operador de asignación y el nuevo valor.

Ejercicios de práctica

1. Declara una variable entera llamada edad y asígnale un valor.

2. Identifica cuáles nombres serían válidos: 2nota, nota_final, promedio, while, mi edad.

3. Explica con tus palabras la diferencia entre declarar una variable e inicializarla.

4. Predice qué valor se mostraría si una variable empieza en 10 y luego recibe el valor 25.

2. Tipos de datos

El tipo de dato determina qué clase de información almacena una variable. Elegirlo correctamente ayuda a representar los datos y a evitar errores.

Tipo	Uso habitual	Ejemplo
`int`	Números enteros	`int edad = 19;`
`double`	Números con decimales y mayor precisión que float en usos comunes	`double pi = 3.14159;`
`float`	Números con decimales, usando normalmente menos memoria que double	`float altura = 1.75f;`
`char`	Un carácter	`char letra = 'M';`
`bool`	Valor lógico verdadero o falso	`bool activo = true;`
`string`	Texto; se usa incluyendo la biblioteca `string`	`string nombre = "Ana";`

Para utilizar `string`, normalmente se incluye `#include <string>`. Para entrada y salida con `cin` y `cout`, se incluye `#include <iostream>`.

Ejemplo

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre = "Luis";
    int edad = 20;
    bool matriculado = true;

    cout << nombre << endl;
    cout << edad << endl;
    cout << matriculado << endl;
    return 0;
}
```

De forma predeterminada, un valor `bool` suele mostrarse como 1 o 0 en la salida estándar. Para imprimir las palabras true o false, se puede usar `boolalpha`.

Conversión de tipos

Una conversión ocurre cuando un valor se transforma de un tipo a otro. Algunas conversiones son automáticas; otras deben indicarse explícitamente. Al convertir un número decimal a un entero, la parte fraccionaria se descarta, no se redondea.

```
double precio = 15.90;
int precioEntero = static_cast<int>(precio);
// precioEntero contiene 15
```

Ejercicios de práctica

1. Escoge un tipo adecuado para guardar: la edad, el precio de un producto, una inicial, un nombre y una respuesta sí/no.
2. Explica qué diferencia existe entre int y double.
3. Predice el resultado de convertir 8.9 a int mediante static_cast.
4. Crea un ejemplo que use al menos cuatro tipos de datos distintos.

3. Asignación de variables

La asignación consiste en guardar un valor en una variable. En C++, el operador básico de asignación es el signo igual (=). La expresión de la derecha se evalúa y su resultado se almacena en la variable de la izquierda.

```
int puntos;
puntos = 10;
puntos = puntos + 5;
// puntos ahora contiene 15
```

La expresión `puntos = puntos + 5` no es una igualdad matemática. En programación, significa: toma el valor actual de puntos, súmale 5 y guarda el resultado de nuevo en puntos.

Declarar, inicializar y reasignar

• Declaración: `int cantidad;`

• Inicialización: `int cantidad = 4;`

• Reasignación: `cantidad = 7;`

Entrada de datos

Con `cin` se puede leer un valor introducido por el usuario y almacenarlo en una variable.

```
int edad;
cout << "Ingrese su edad: ";
cin >> edad;
cout << "Edad registrada: " << edad << endl;
```

Antes de usar una variable para un cálculo, asegúrate de que contenga un valor válido. Una variable local incorporada a una función no debe utilizarse sin inicializarla.

Ejercicios de práctica

1. Traza los valores de x después de ejecutar: int x = 4; x = x + 3; x = x * 2.
2. Escribe los pasos para leer dos números y guardar su suma en una tercera variable.
3. Explica por qué x = x + 1 es una instrucción válida en C++.
4. Crea un programa que lea el precio y la cantidad de un artículo y calcule el total.

4. Constantes

Una constante representa un valor que no debe cambiar después de inicializarse. En C++ se puede declarar con la palabra clave `const`.

```
const double IVA = 0.13;
const int DIAS_SEMANA = 7;
```

Después de inicializar `IVA`, una asignación como `IVA = 0.15;` provoca un error de compilación. Por convención, las constantes suelen escribirse con mayúsculas y guion bajo, aunque el lenguaje no obliga a hacerlo.

¿Cuándo conviene usar constantes?

- Cuando un valor representa una regla que no debe cambiar durante el programa.
- Para evitar repetir números fijos en muchas partes del código.
- Para mejorar la legibilidad: `IVA` es más descriptivo que un número sin contexto.

Ejemplo

```
#include <iostream>
using namespace std;

int main() {
    const double PRECIO = 1250.0;
    int cantidad = 4;
    double total = PRECIO * cantidad;

    cout << "Total: " << total << endl;
    return 0;
}
```

Ejercicios de práctica

1. Declara una constante para representar los minutos de una hora.
2. Explica qué ocurre si intentas modificar una constante después de inicializarla.
3. Diseña un cálculo de área de un círculo usando una constante para pi.
4. Indica cuándo es preferible usar una constante en vez de una variable.

5. Operadores

Los operadores permiten realizar cálculos, comparar valores, combinar condiciones y modificar variables. Los operadores se agrupan según la tarea que realizan.

5.1. Operadores matemáticos

Operador	Significado	Ejemplo
+	Suma	`7 + 3` produce 10
-	Resta	`7 - 3` produce 4
*	Multiplicación	`7 * 3` produce 21
/	División	`7.0 / 2` produce 3.5
%	Residuo de división entera	`7 % 3` produce 1

Atención: si ambos operandos de una división son enteros, el resultado también será entero. Por ejemplo, `7 / 2` produce 3, mientras que `7.0 / 2` produce 3.5.

El operador módulo (%) devuelve el residuo y se utiliza con operandos enteros. Puede ayudar a comprobar si un número es par: si `numero % 2 == 0`, el residuo es cero.

Ejemplo

```
int a = 17;
int b = 5;
cout << a + b << endl; // 22
cout << a / b << endl; // 3
cout << a % b << endl; // 2
```

5.2. Operadores de asignación

Además de =, C++ incluye operadores que combinan una operación con la asignación del resultado.

Operador	Equivalencia conceptual	Ejemplo si x = 10
+=	x = x + valor	x += 3 → 13
-=	x = x - valor	x -= 3 → 7
*=	x = x * valor	x *= 2 → 20
/=	x = x / valor	x /= 2 → 5
%=	x = x % valor	x %= 3 → 1

La división y el módulo tienen restricciones relacionadas con los tipos enteros cuando se utilizan sus versiones compuestas. Evita dividir entre cero.

5.3. Operadores relacionales

Los operadores relacionales comparan dos valores. El resultado de una comparación es un valor booleano: verdadero (true) o falso (false).

Operador	Significado	Ejemplo
==	Igual a	`edad == 18`
!=	Distinto de	`edad != 18`
>	Mayor que	`nota > 70`
<	Menor que	`nota < 70`
>=	Mayor o igual que	`nota >= 70`
<=	Menor o igual que	`nota <= 70`

No confundas = con ==. El primero asigna un valor; el segundo compara dos valores.

```
int nota = 75;
bool aprobo = nota >= 70;
// aprobo contiene true
```

5.4. Operadores lógicos

Los operadores lógicos permiten combinar o negar condiciones. Son muy usados en estructuras condicionales y ciclos.

Operador	Interpretación	Cuándo resulta verdadero
&&	AND lógico (y)	Cuando ambas condiciones son verdaderas
||	OR lógico (o)	Cuando al menos una condición es verdadera

Operador	Interpretación	Cuándo resulta verdadero
!	NOT lógico (no)	Invierte el valor lógico

```
int edad = 20;
bool tieneEntrada = true;

bool puedeEntrar = (edad >= 18) && tieneEntrada;
bool necesitaRevision = (edad < 18) || !tieneEntrada;
```

En el primer caso, ambas condiciones deben cumplirse: tener al menos 18 años y contar con entrada. En el segundo, basta con que una condición sea verdadera.

Ejercicios de práctica

1. Calcula manualmente el resultado de 19 % 4 y explica qué representa.
2. Si x vale 12, determina el resultado final de x += 5 y luego x /= 2.
3. Escribe una comparación que compruebe si una edad es menor que 18.
4. Formula una condición lógica que indique que una nota está entre 70 y 100, inclusive.
5. Explica la diferencia entre && y || mediante un ejemplo propio.
6. Identifica el error conceptual en la expresión nota = 70 cuando se pretende comprobar si nota vale 70.

6. Ejercicios integradores

Resuelve los siguientes problemas utilizando variables, tipos de datos, asignación, constantes y operadores. Antes de programar, identifica los datos de entrada, el proceso y el resultado esperado.

Nivel 1: fundamentos

1. Datos personales: declara variables para nombre, edad y una inicial. Muestra sus valores en pantalla.
2. Suma básica: solicita dos números enteros y muestra la suma, resta y multiplicación.
3. Conversión: lee una cantidad de minutos e indica cuántas horas completas representa y cuántos minutos sobran. Pista: utiliza división entera y módulo.

Nivel 2: cálculos

1. Compra: solicita el precio unitario y la cantidad de productos. Calcula el subtotal.
2. Promedio: lee tres calificaciones y calcula el promedio usando un tipo apropiado.
3. Área: declara una constante para pi, solicita el radio y calcula el área de un círculo.

Nivel 3: comparaciones y lógica

1. Par o impar: lee un número entero y formula una condición con el operador módulo para comprobar si es par.
2. Mayor de edad: solicita una edad y construye una expresión lógica que indique si la persona tiene 18 años o más.
3. Rango de nota: solicita una calificación y comprueba si está entre 0 y 100, inclusive.
4. Descuento: solicita el total de una compra y el estado de cliente frecuente. Diseña una condición lógica para decidir si corresponde aplicar un descuento según reglas que definas y documentes.

Método recomendado para resolver

• Comprende el enunciado y subraya qué datos se reciben y qué resultado se pide.

• Elige el tipo de dato para cada variable.

• Escribe la fórmula o condición en lenguaje cotidiano antes de traducirla a C++.

• Prueba con valores sencillos y con casos límite.

• Revisa nombres, tipos de datos, operadores y posibles divisiones entre cero.

7. Buenas prácticas y errores comunes

Buenas prácticas

• Usa nombres que expliquen el propósito de cada variable.

• Inicializa las variables antes de leerlas o utilizarlas.

• Usa `const` para valores que no deberían cambiar.

• Agrega comentarios cuando aclaren una decisión; evita comentar lo obvio.

• Separa las expresiones complejas con paréntesis cuando mejoren la lectura.

• Comprueba los límites de entrada y evita operaciones no válidas, como dividir entre cero.

Errores frecuentes

• Confundir = y ==: asignar no es lo mismo que comparar.

• División entera: dividir dos enteros puede descartar la parte decimal.

• Tipo inadecuado: guardar texto en una variable numérica no funciona como se espera.

• Variable sin inicializar: usar una variable local sin un valor definido puede producir resultados no fiables.

• Nombre inválido: los espacios y las palabras reservadas no son válidos en identificadores.

• Olvidar bibliotecas: usa las cabeceras necesarias, como `<iostream>` y `<string>`.

Lista de comprobación

☐ Puedo explicar qué es una variable y para qué sirve.

☐ Sé distinguir int, double, char, bool y string.

☐ Puedo declarar, inicializar y reasignar una variable.

☐ Comprendo cuándo conviene usar const.

☐ Puedo diferenciar operadores matemáticos, de asignación, relacionales y lógicos.

☐ Puedo seguir manualmente la ejecución de una expresión.

Glosario breve

Término	Definición
Asignación	Operación que guarda un valor en una variable.
Booleano	Tipo de dato lógico con valores true o false.
Compilador	Programa que traduce el código fuente a una forma ejecutable.
Constante	Identificador cuyo valor no puede reasignarse después de inicializarse.
Declaración	Instrucción que introduce una variable indicando su tipo y nombre.
Inicialización	Asignación del valor inicial de una variable.
Operando	Valor o variable sobre el que actúa un operador.
Residuo	Resto que queda tras una división entera.

Fin de la guía. La mejor forma de aprender estos fundamentos es predecir primero la salida de un programa, ejecutarlo y explicar cualquier diferencia.